

ITA0504-COMPUTER VISION

CO1-TOOL1-PYTHON

QUESTION 1

1. Explain the effect of sampling on image quality. Demonstrate the impact of improper sampling on an image and apply suitable techniques to improve the image quality using a Python program.

Python Program

```
from google.colab import files
import cv2
import matplotlib.pyplot as plt

uploaded = files.upload()
filename = next(iter(uploaded))

# Read image
image = cv2.imread(filename)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Improper sampling
small = cv2.resize(image, None, fx=0.15, fy=0.15,
interpolation=cv2.INTER_AREA)

improper = cv2.resize(
    small,
    (image.shape[1], image.shape[0]),
    interpolation=cv2.INTER_NEAREST
)
```

```
# Improve image quality
corrected = cv2.resize(
    small,
    (image.shape[1], image.shape[0]),
    interpolation=cv2.INTER_CUBIC
)

blur = cv2.GaussianBlur(corrected, (0, 0), 1.2)
corrected = cv2.addWeighted(corrected, 1.5, blur, -0.5, 0)

# Display images
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(improper)
plt.title("Improper Sampling")
plt.axis("off")

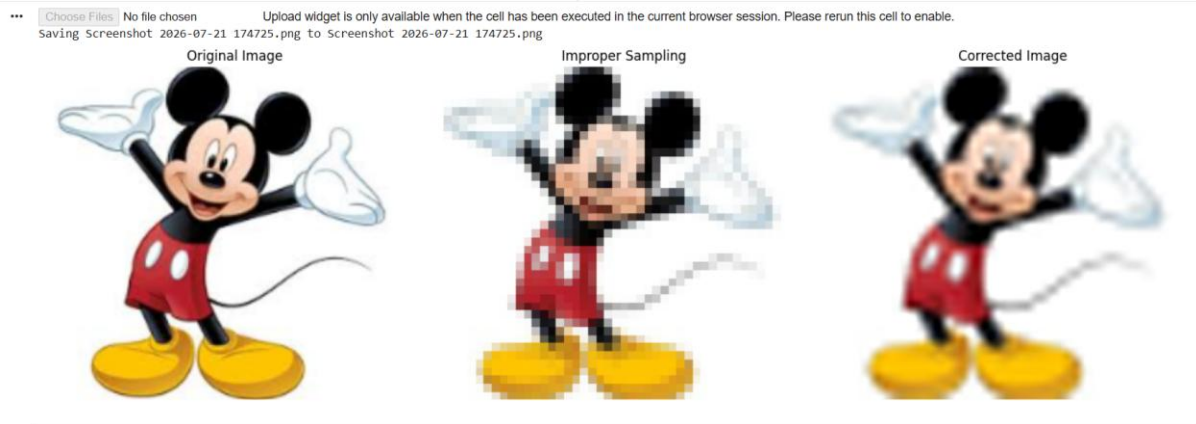
plt.subplot(1,3,3)
plt.imshow(corrected)
plt.title("Corrected Image")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT:



QUESTION 2

2. An image is acquired under low-light conditions, resulting in poor visibility. Explain how image acquisition affects image quality and demonstrate techniques to improve the image using a Python program.

Python Program

```
from google.colab import files
```

```
import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
uploaded = files.upload()
```

```
filename = next(iter(uploaded))
```

```
# Read image
```

```
image = cv2.imread(filename)
```

```
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Create low-light image
low_light = cv2.convertScaleAbs(image, alpha=0.4, beta=-20)

# Gamma correction
gamma = 0.5

table = np.array([
    ((i / 255.0) ** gamma) * 255
    for i in np.arange(256)
]).astype("uint8")

bright_image = cv2.LUT(low_light, table)

# Contrast enhancement using CLAHE
lab = cv2.cvtColor(bright_image, cv2.COLOR_RGB2LAB)
l, a, b = cv2.split(lab)

clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8,8)
)

l = clahe.apply(l)

enhanced_image = cv2.merge((l, a, b))
```

```
enhanced_image = cv2.cvtColor(  
    enhanced_image,  
    cv2.COLOR_LAB2RGB  
)
```

```
# Display images  
plt.figure(figsize=(15,5))
```

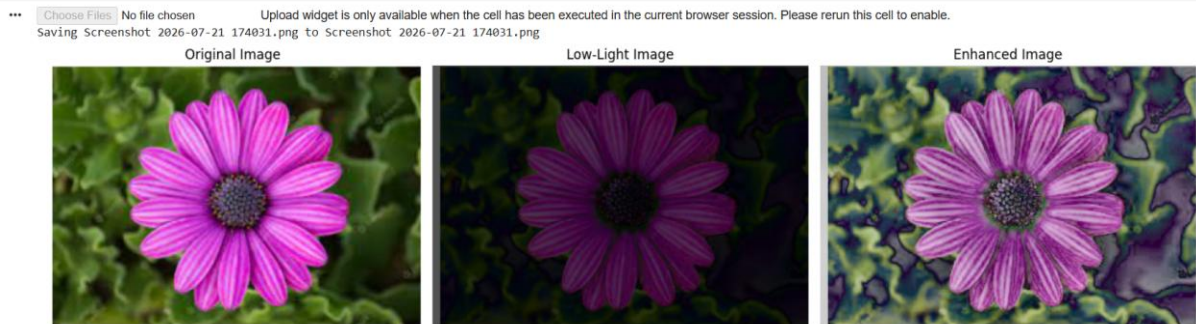
```
plt.subplot(1,3,1)  
plt.imshow(image)  
plt.title("Original Image")  
plt.axis("off")
```

```
plt.subplot(1,3,2)  
plt.imshow(low_light)  
plt.title("Low-Light Image")  
plt.axis("off")
```

```
plt.subplot(1,3,3)  
plt.imshow(enhanced_image)  
plt.title("Enhanced Image")  
plt.axis("off")
```

```
plt.tight_layout()  
plt.show()
```

OUTPUT:



QUESTION 3

3. Explain the concept of aliasing in digital images. Demonstrate the effect of aliasing due to improper sampling and apply a suitable technique to reduce aliasing using a Python program.

Python Program

```
from google.colab import files
import cv2
import matplotlib.pyplot as plt

uploaded = files.upload()
filename = next(iter(uploaded))

# Read image
image = cv2.imread(filename)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Create aliasing effect
small_image = cv2.resize(
    image,
    None,
    fx=0.2,
```

```
    fy=0.2,  
    interpolation=cv2.INTER_NEAREST  
)
```

```
aliased_image = cv2.resize(  
    small_image,  
    (image.shape[1], image.shape[0]),  
    interpolation=cv2.INTER_NEAREST  
)
```

```
# Reduce aliasing  
filtered_image = cv2.GaussianBlur(image, (5, 5), 0)
```

```
small_filtered = cv2.resize(  
    filtered_image,  
    None,  
    fx=0.2,  
    fy=0.2,  
    interpolation=cv2.INTER_AREA  
)
```

```
corrected_image = cv2.resize(  
    small_filtered,  
    (image.shape[1], image.shape[0]),  
    interpolation=cv2.INTER_CUBIC  
)
```

```
# Display images
```

```
plt.figure(figsize=(15,5))
```

```
plt.subplot(1,3,1)
```

```
plt.imshow(image)
```

```
plt.title("Original Image")
```

```
plt.axis("off")
```

```
plt.subplot(1,3,2)
```

```
plt.imshow(aliased_image)
```

```
plt.title("Aliased Image")
```

```
plt.axis("off")
```

```
plt.subplot(1,3,3)
```

```
plt.imshow(corrected_image)
```

```
plt.title("Corrected Image")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT:



QUESTION 4

4. Explain the concepts of pixel resolution and intensity resolution in digital images. Demonstrate their effects on image quality using a Python program.

Python Program

```
from google.colab import files
import cv2
import matplotlib.pyplot as plt

uploaded = files.upload()
filename = next(iter(uploaded))

# Read image in grayscale
image = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)

# Low pixel resolution
low_pixel = cv2.resize(
    image,
    (100, 100),
    interpolation=cv2.INTER_AREA
)

low_pixel = cv2.resize(
    low_pixel,
    (image.shape[1], image.shape[0]),
    interpolation=cv2.INTER_NEAREST
)
```

```
# Intensity resolution
two_bit = (image // 64) * 64
four_bit = (image // 16) * 16
eight_bit = image

# Display images
plt.figure(figsize=(16,5))

plt.subplot(1,4,1)
plt.imshow(low_pixel, cmap="gray")
plt.title("Low Pixel Resolution")
plt.axis("off")

plt.subplot(1,4,2)
plt.imshow(two_bit, cmap="gray")
plt.title("2-Bit Intensity")
plt.axis("off")

plt.subplot(1,4,3)
plt.imshow(four_bit, cmap="gray")
plt.title("4-Bit Intensity")
plt.axis("off")

plt.subplot(1,4,4)
plt.imshow(eight_bit, cmap="gray")
plt.title("8-Bit Intensity")
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT:



QUESTION 5

5. A real-time surveillance system is used to monitor an area. Explain the levels of Computer Vision involved in this application. Write a Python program to perform basic video processing by reading a video and displaying its frames.

Python Program

```
import cv2
```

```
import matplotlib.pyplot as plt
```

```
video = cv2.VideoCapture("video.mp4")
```

```
frame_count = 0
```

```
while True:
```

```
    ret, frame = video.read()
```

```
    if not ret:
```

```
        break
```

```
    frame_count += 1
```

```
if frame_count % 30 == 0:
```

```
    frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

```
    plt.figure(figsize=(6,4))
```

```
    plt.imshow(frame)
```

```
    plt.title(f'Frame {frame_count}')
```

```
    plt.axis("off")
```

```
    plt.show()
```

```
video.release()
```

```
print("Video Processing Completed Successfully")
```

OUTPUT:



Frame 270



QUESTION 6

6. Explain the image formation model in Computer Vision. Demonstrate image degradation and enhancement using a Python program.

Python Program

```
from google.colab import files
import cv2
import matplotlib.pyplot as plt

uploaded = files.upload()
filename = next(iter(uploaded))

# Read image
image = cv2.imread(filename)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Create degraded image
degraded = cv2.GaussianBlur(image, (9, 9), 0)
degraded = cv2.convertScaleAbs(degraded, alpha=0.6, beta=-20)

# Enhance image
enhanced = cv2.equalizeHist(cv2.cvtColor(degraded,
cv2.COLOR_RGB2GRAY))

# Display images
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(image)
```

```
plt.title("Original Image")
```

```
plt.axis("off")
```

```
plt.subplot(1,3,2)
```

```
plt.imshow(degraded)
```

```
plt.title("Degraded Image")
```

```
plt.axis("off")
```

```
plt.subplot(1,3,3)
```

```
plt.imshow(enhanced, cmap="gray")
```

```
plt.title("Enhanced Image")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT:



QUESTION 7

7. Explain the role of quantization in converting a grayscale image into a digital image. Demonstrate different quantization levels using a Python program.

Python Program

```
from google.colab import files
```

```
import cv2
```

```
import matplotlib.pyplot as plt
```

```
uploaded = files.upload()
filename = next(iter(uploaded))

# Read grayscale image
image = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)

# Quantization
level_2 = (image // 128) * 128
level_4 = (image // 64) * 64
level_16 = (image // 16) * 16
original = image

# Display images
plt.figure(figsize=(16,5))

plt.subplot(1,4,1)
plt.imshow(level_2, cmap="gray")
plt.title("2 Levels")
plt.axis("off")

plt.subplot(1,4,2)
plt.imshow(level_4, cmap="gray")
plt.title("4 Levels")
plt.axis("off")

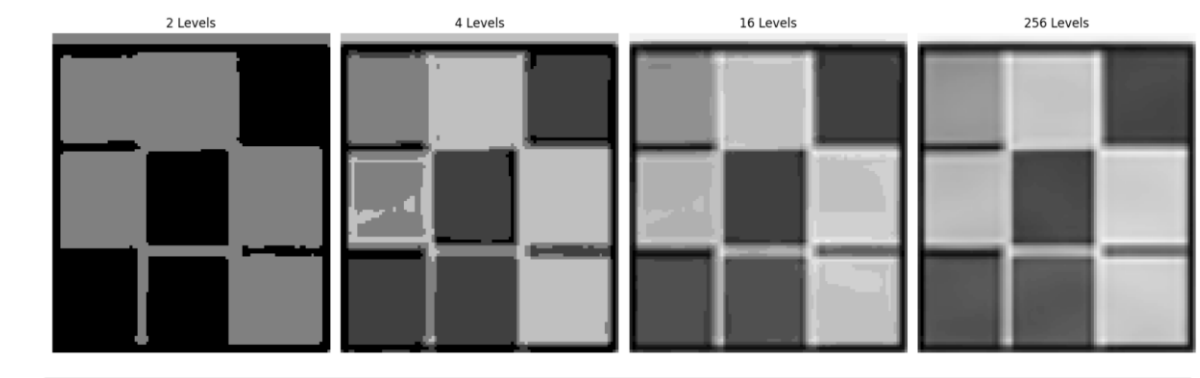
plt.subplot(1,4,3)
```

```
plt.imshow(level_16, cmap="gray")
plt.title("16 Levels")
plt.axis("off")
```

```
plt.subplot(1,4,4)
plt.imshow(original, cmap="gray")
plt.title("Original (256 Levels)")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

OUTPUT:



QUESTION 8

8. Explain the effect of sensor noise on digital images. Demonstrate the addition of noise and its removal using a Python program.

Python Program

```
from google.colab import files
import cv2
import numpy as np
import matplotlib.pyplot as plt

uploaded = files.upload()
```



```
filename = next(iter(uploaded))

# Read image
image = cv2.imread(filename)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Add Gaussian noise
noise = np.random.normal(0, 25, image.shape)

noisy_image = image.astype(np.float32) + noise
noisy_image = np.clip(noisy_image, 0, 255).astype(np.uint8)

# Remove noise using Gaussian Blur
denoised_image = cv2.GaussianBlur(
    noisy_image,
    (5, 5),
    0
)

# Display images
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")
```

```
plt.subplot(1,3,2)
plt.imshow(noisy_image)
plt.title("Noisy Image")
plt.axis("off")
```

```
plt.subplot(1,3,3)
plt.imshow(denoised_image)
plt.title("Denoised Image")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

OUTPUT:



QUESTION 9

9. Explain the different levels of Computer Vision involved in facial recognition. Write a Python program to detect faces in an image using OpenCV.

Python Program

```
from google.colab import files
import cv2
import matplotlib.pyplot as plt
```

```
uploaded = files.upload()
filename = next(iter(uploaded))
```

```
# Read image

image = cv2.imread(filename)

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)


# Load Haar Cascade classifier

face_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades + "haarcascade_frontalface_default.xml"
)


# Detect faces

faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5,
    minSize=(30,30)
)


# Draw rectangle around faces

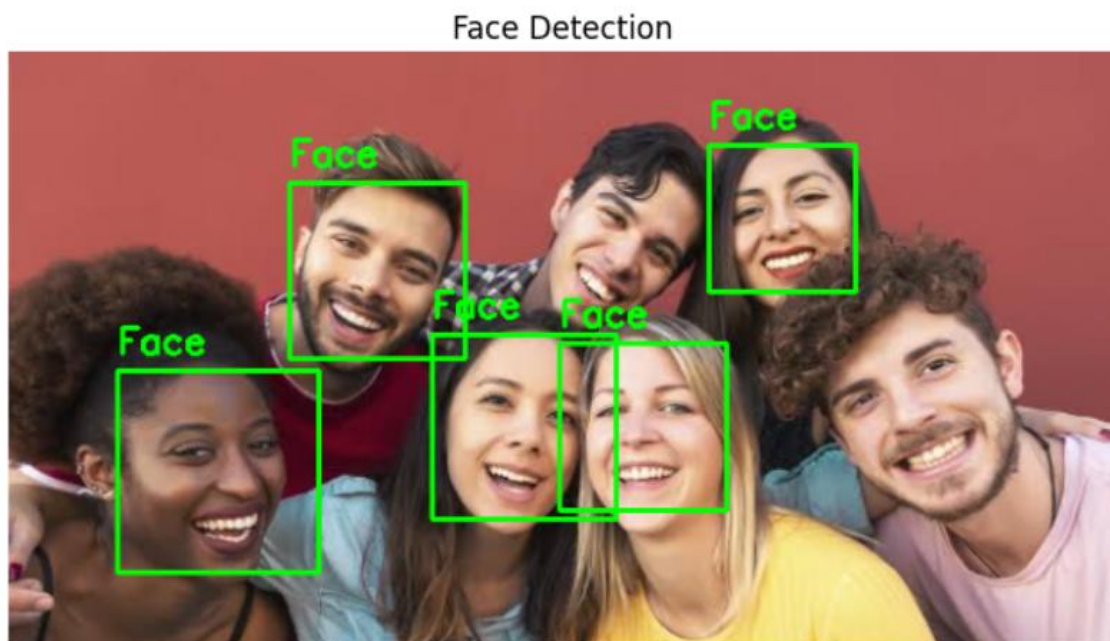
for (x, y, w, h) in faces:
    cv2.rectangle(
        image,
        (x, y),
        (x+w, y+h),
        (0,255,0),
        2
    )
```

```
# Convert to RGB for display
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Display output
plt.figure(figsize=(8,8))
plt.imshow(image)
plt.title("Face Detection")
plt.axis("off")
plt.show()

print("Number of Faces Detected:", len(faces))
```

OUTPUT:



QUESTION 10

10. A grayscale image is converted into a digital image. Describe the role of quantization in this process and its impact on image detail. Write a Python program to demonstrate quantization of a grayscale image.

Python Program

```
from google.colab import files
import cv2
import matplotlib.pyplot as plt

uploaded = files.upload()
filename = next(iter(uploaded))

# Read image in grayscale
image = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)

# Quantization with different intensity levels
level_2 = (image // 128) * 128
level_4 = (image // 64) * 64
level_8 = (image // 32) * 32
original = image

# Display images
plt.figure(figsize=(16,5))

plt.subplot(1,4,1)
plt.imshow(level_2, cmap="gray")
plt.title("2 Levels")
plt.axis("off")
```

```
plt.subplot(1,4,2)
plt.imshow(level_4, cmap="gray")
plt.title("4 Levels")
plt.axis("off")
```

```
plt.subplot(1,4,3)
plt.imshow(level_8, cmap="gray")
plt.title("8 Levels")
plt.axis("off")
```

```
plt.subplot(1,4,4)
plt.imshow(original, cmap="gray")
plt.title("Original (256 Levels)")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

OUTPUT:

